Lab 04 — Change Management with Version-Controlled Configuration

> **SY0-701 Objective 1.3** — Explain the importance of change management processes and the impact to security.
>
> **Domain 1** — General Security Concepts (12%) · **Time:** ~45 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Take one security change through the full change management process — request, impact analysis, test, approval, implementation in a maintenance window, documentation — then detect an unauthorised change and back it out.

You'll build: A change request record (CHG-1001), a staging test with recorded results, an nginx allow list and deny list applied with a zero-downtime reload, a version-controlled `/etc` (etckeeper + git) with the change committed, updated documentation, and an executed backout of an unapproved edit.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; tools installed in step 1.

Background

- Unplanned and unauthorised changes are a leading cause of outages *and* of new vulnerabilities, so change management is a security control, not just paperwork.
- **Business processes**: approval process (change advisory board), ownership, stakeholders, impact analysis, test results, backout plan, maintenance window and standard operating procedures (SOPs).
- **Technical implications**: allow lists and deny lists that must be updated, restricted activities (what is in scope), downtime, service and application restarts, legacy applications and dependencies.
- **Documentation**: update diagrams and policies/procedures when a change closes, and keep every configuration under **version control** so you can see who changed what and roll back.
- A **reload** applies new configuration without dropping connections; a **restart** stops and starts the service, causing brief downtime.

Steps

1. Install the tools

etckeeper puts `/etc` into a git repository and auto-commits every package install. nginx is the production service you will change. Set a git identity first so commits are attributed.

```
apt-get update -qq && apt-get install -y git curl
git config --global user.name "Lab Admin" && git config --global user.email
"admin@lab.local"
DEBIAN_FRONTEND=noninteractive apt-get install -y etckeeper nginx
systemctl enable --now nginx
mkdir -p /root/change/staging && cd /root/change
```

2. Put /etc under version control and record the baseline

A known baseline is the reference point for detecting drift. etckeeper already committed the nginx installation for you.

```
[ -d /etc/.git ] || etckeeper init
etckeeper commit "Baseline before CHG-1001" >/dev/null 2>&1 || true
git -C /etc log --oneline | head -5
git -C /etc status --short | wc -l
```

The status count should be `0`: the running configuration matches the last commit.

3. Raise the change request

The request names an owner and stakeholders, states what is restricted (out of scope), and contains a backout plan *before* anyone approves it.

```
cat > /root/change/CHG-1001.txt <<'EOF'
CHANGE REQUEST CHG-1001
Title        : Restrict nginx admin port 8081 to an allow list, add a deny list and
nosniff header
Owner        : Lab Admin (web platform team)
Stakeholders : Web users, security operations, monitoring team, service desk
Type         : Normal change (CAB review required)
Reason       : Pen-test finding PT-17 - admin interface reachable from any address
Scope        : /etc/nginx/conf.d/chg-1001.conf on web01 only
Restricted   : No edits outside conf.d, no restart of other services, no package upgrades
Impact       : Admin port closed to all except allow list; config reload only (no
downtime)
Dependencies : Monitoring health check polls 127.0.0.1:8081 every minute; legacy report
tool uses port 80
Window       : Sunday 02:00-03:00 SGT (lowest traffic)
Test plan    : Staging instance; nginx -t; allow-listed source gets 200; header present
Backout plan : git -C /etc revert the CHG-1001 commit, nginx -t, systemctl reload nginx
Approval     : PENDING CAB
EOF
cat /root/change/CHG-1001.txt
```

4. Run the impact analysis

Check what is listening, what depends on nginx, and which files the change will touch. Append the findings to the change record.

```
ss -tlnp | grep nginx
systemctl list-dependencies --reverse nginx.service --no-pager | head -5
ls /etc/nginx/conf.d/ /etc/nginx/sites-enabled/
grep -rn "listen" /etc/nginx/sites-enabled/
echo "Impact check $(date +%F): nginx on :80 only today; new :8081 listener; monitoring at
127.0.0.1 is a dependency" >> /root/change/CHG-1001.txt
```

5. Stage and test the change

Write the candidate configuration and test it on a separate staging nginx instance, not on production. The first test exposes a dependency problem: the allow list would block the monitoring host.

```
cat > /root/change/staging/chg-1001.conf <<'EOF'
# CHG-1001: admin allow list, deny list and security header
deny 203.0.113.0/24;
deny 198.51.100.23;
add_header X-Content-Type-Options "nosniff" always;
server {
    listen 8081;
    root /var/www/html;
    index index.html index.nginx-debian.html;
    location / {
        allow 10.10.0.0/16;
        deny  all;
    }
}
EOF
cat > /root/change/staging/nginx.conf <<'EOF'
pid /root/change/staging/nginx.pid;
error_log /root/change/staging/error.log;
events {}
http { access_log off; include /root/change/staging/chg-1001.conf; }
EOF
nginx -t -p /root/change/staging -c /root/change/staging/nginx.conf
nginx -p /root/change/staging -c /root/change/staging/nginx.conf && sleep 1
curl -s -o /dev/null -w "staging test 1 (monitoring host): HTTP %{http_code}\n"
http://127.0.0.1:8081/
sed -i 's#allow 10.10.0.0/16;#allow 10.10.0.0/16;\n        allow 127.0.0.1;#'
/root/change/staging/chg-1001.conf
nginx -p /root/change/staging -c /root/change/staging/nginx.conf -s reload && sleep 1
curl -s -o /dev/null -w "staging test 2 (monitoring host): HTTP %{http_code}\n"
http://127.0.0.1:8081/
nginx -p /root/change/staging -c /root/change/staging/nginx.conf -s stop
echo "Test results $(date +%T): test 1 HTTP 403 (monitoring blocked), allow 127.0.0.1
added, test 2 HTTP 200" >> /root/change/CHG-1001.txt
```

```
staging test 1 (monitoring host): HTTP 403
staging test 2 (monitoring host): HTTP 200
```

6. Approve and implement in the maintenance window

With test results attached, the CAB approves. Deploy the tested file, validate it, and apply it with a **reload**: the nginx master PID stays the same, so existing connections are not dropped.

```
sed -i "s/^Approval     : PENDING CAB/Approval     : APPROVED by CAB on $(date +%F) for
the Sunday window/" /root/change/CHG-1001.txt
grep '^Approval' /root/change/CHG-1001.txt
cp /root/change/staging/chg-1001.conf /etc/nginx/conf.d/chg-1001.conf
nginx -t
OLD=$(cat /run/nginx.pid); systemctl reload nginx; sleep 1; NEW=$(cat /run/nginx.pid)
echo "nginx master PID before=$OLD after=$NEW (same PID = reload, no downtime)"
curl -s -o /dev/null -w "production admin port from monitoring host: HTTP %{http_code}\n"
```

```
http://127.0.0.1:8081/
curl -sI http://127.0.0.1/ | grep -i "x-content-type-options"
```

7. Commit the change and update the documentation

Commit with the change ID so the history links back to the request, then update the network diagram and the change log. Version control answers "who changed what, when, and why".

```
etckeeper commit "CHG-1001: nginx admin allow list, deny list, nosniff header"
git -C /etc log --oneline -3
git -C /etc show --stat HEAD | head -12
cat >> /root/change/network-diagram.txt <<'EOF'
[Internet] --80/tcp--> [web01 nginx]   (deny list: 203.0.113.0/24, 198.51.100.23)
[Admin VLAN 10.10.0.0/16 + monitoring 127.0.0.1] --8081/tcp--> [web01 admin]   (updated
CHG-1001)
EOF
echo "$(date +%F) CHG-1001 implemented and verified; SOP-WEB-02 updated with the allow-
list request step" >> /root/change/CHANGELOG.txt
cat /root/change/network-diagram.txt /root/change/CHANGELOG.txt
```

8. Detect an unauthorised change and execute the backout plan

Someone edits production outside the process and breaks the syntax. Version control shows the drift instantly, the config test fails, and you restore the approved version.

```
echo "deny 127.0.0.1" >> /etc/nginx/conf.d/chg-1001.conf
git -C /etc status --short nginx/
git -C /etc diff nginx/conf.d/chg-1001.conf | tail -3
nginx -t 2>&1 | tail -1 || true
git -C /etc checkout -- nginx/conf.d/chg-1001.conf
nginx -t && systemctl reload nginx
git -C /etc status --short nginx/ | wc -l
echo "$(date +%F) Unauthorised edit to chg-1001.conf detected and reverted; incident INC-
2001 raised" >> /root/change/CHANGELOG.txt
```

The final count should be `0`: production matches the approved, committed configuration again.

Test it

```
git -C /etc log --oneline | grep -c "CHG-1001:"
git -C /etc status --short nginx/ | wc -l
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8081/
curl -sI http://127.0.0.1/ | grep -ci "x-content-type-options: nosniff"
grep -c "APPROVED" /root/change/CHG-1001.txt
```

Expected: `1` (the CHG-1001 commit), `0` (no drift), `200`, `1` (header present) and `1` (approval recorded).

Troubleshooting

Symptom	Fix
`etckeeper commit` fails with `Please tell me who you are`	Run the two `git config --global` commands from step 1, then repeat the commit.
Staging nginx fails with `bind() to 0.0.0.0:8081 failed (98: Address already in use)`	A previous staging instance is still running. Stop it with `nginx -p /root/change/staging -c /root/change/staging/nginx.conf -s stop`, or check `ss -tlnp | grep 8081`.
`cat: /run/nginx.pid: No such file or directory`	nginx is not running; start it with `systemctl start nginx` and check `systemctl status nginx --no-pager`.

Challenge

Execute the **full backout plan** from the change record: `git -C /etc revert --no-edit HEAD`, then `nginx -t && systemctl reload nginx`, and prove port 8081 is closed again. Then write a one-page **SOP** for "standard change: add an IP to the nginx deny list" so that this routine change can be pre-approved without a full CAB review.

Reflection

The first staging test returned 403 for the monitoring host. What would have happened if the change had gone straight to production without testing, and which part of the change process was designed to catch that?

Exam tips

- **Backout plan** = how to return to the last known-good state; it is prepared and approved *before* implementation.
- **Impact analysis** happens before approval and covers the risk of making the change *and* of not making it.
- **Dependencies** and **legacy applications** are the usual reasons a small change causes an outage; restarts mean downtime, reloads usually do not.
- After implementation, update **diagrams, policies and procedures**, and keep configs in **version control** to detect drift and unauthorised changes.

References

- NIST SP 800-128 Security-Focused Configuration Management — https://csrc.nist.gov/pubs/sp/800/128/upd1/final
- NIST SP 800-53 CM-3 Configuration Change Control — https://csrc.nist.gov/projects/risk-management/sp800-53-controls/release-search
- etckeeper documentation — https://etckeeper.branchable.com/
- nginx access module (allow/deny) — https://nginx.org/en/docs/http/ngx_http_access_module.html

---